Neural Network Learning of Black-Scholes Equation for Option Pricing

A presentation on De Souza Santos and Ferreira's (2024) paper
MH4110 Partial Differential Equations

Ben Henderson (N2504294J), Frederick Nathanael Ginting (U2340564B)
Kiang Choon Kiat Keith (U2240202A), Wilbert Thamrin (U2340880C)

School of Physical and Mathematical Sciences, Nanyang Technological University

Thursday, 26th March 2026

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

1. Black-Scholes Equation
2. Neural Network Learning
3. Experimentation and Results
4. Evaluation and Conclusion

1 Black-Scholes Equation
What is an Option?
Black-Scholes Equation
Transformation to the Heat Equation

2 Neural Network Learning

3 Experimentation and Results

4 Evaluation and Conclusion

1 Black-Scholes Equation
What is an Option?
Black-Scholes Equation
Transformation to the Heat Equation

2 Neural Network Learning

3 Experimentation and Results

4 Evaluation and Conclusion

What is an Option?

- Derivatives are financial contracts which derive their value from an underlying asset
- Futures, Forwards, Swaps, Options
- Options:
  - Gives the right, not the obligation, to buy/sell asset at specified price on specified date
  - Call gives right to buy
  - Put gives right to sell
  - American style can be utilised at any time up until expiration date
  - European style can only be utilised on expiration date

1 Black-Scholes Equation
What is an Option?
Black-Scholes Equation
Transformation to the Heat Equation

2 Neural Network Learning

3 Experimentation and Results

4 Evaluation and Conclusion

Black-Scholes Background

- Developed in 1970s by Fischer Black, Myron Scholes, and Robert Merton
- Provides an analytic solution to a second order parabolic partial differential equation by a transformation to the heat equation
- Leads to conclusion that options pricing dynamic is similar to heat dissipation
- Based on European style options
- American style options pricing can only be done numerically, not analytically

Black-Scholes Equation

The Black-Scholes equation is a second-order parabolic PDE. For a stock which does not pay dividends, we have:

$$\frac{\partial c}{\partial t} + rS\frac{\partial c}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 c}{\partial S^2} = rc$$

- t = time
- c = call option price
- p = put option price
- S = spot stock price
- r = risk free interest rate
- σ = stock volatility
- S_0 = spot stock price at time zero
- K = strike price
- T = time to maturity
- $N(x)$ = Cumulative Distribution Probability

Solutions to this PDE are the formulas for European call and put options prices.

Boundary Conditions

- Boundary conditions come from the payoff, given by:

$$\text{payoff} = \text{strike} - \text{spot } S_t$$

- For a European call option, we have the terminal condition:

$$\text{payoff} = \max\{S_T - K, 0\}$$

- So we have the lower bound of zero for $S = 0$, and the upper bound of S as $S \to \infty$

1 Black-Scholes Equation
What is an Option?
Black-Scholes Equation
Transformation to the Heat Equation

2 Neural Network Learning

3 Experimentation and Results

4 Evaluation and Conclusion

Transformation to the Heat Equation

Firstly, we define:

$$x = \ln\left(\frac{S(t)}{c(t)}\right) \qquad \text{and} \qquad \tau = \frac{1}{2}\sigma^2(T-t)$$

Take log of price as stock price follows geometric Brownian motion.
Define rescale of time as time scaled by volatility.
Define mapping from $c(s,t)$ to $f(x,\tau)$ with strike price K as scale factor:

$$c(s,t) = Kf(x,\tau)$$

Substituting in the change of variables and mappings so far allows us to find the PDE:

$$\frac{\partial f}{\partial \tau} = \frac{\partial^2 f}{\partial x^2} + \left(\frac{2r}{\sigma^2} - 1\right)\frac{\partial f}{\partial x} - \frac{2r}{\sigma^2} f$$

which can be re-written as:

$$\frac{\partial f}{\partial \tau} = \frac{\partial^2 f}{\partial x^2} + (k-1)\frac{\partial f}{\partial x} - kf$$

where $k = \frac{2r}{\sigma^2}$. We see this is a PDE with similar form to the heat equation, but with extra terms $(k-1)\frac{\partial f}{\partial x}$ and $-kf$.

To make the extra terms disappear, we define a new mapping, where the exponential removes drift:

$$f(x,\tau) = \exp(\alpha x + \beta\tau)\, u(x,\tau)$$

We then solve and choose α and β such that the extra terms are removed, so we find:

$$\alpha = -\tfrac{1}{2}(k-1) \qquad \text{and} \qquad \beta = -\tfrac{1}{4}(k+1)^2$$

which leads to a direct relationship between actual price of the option $c(s,t)$, and the modified price $u(x,\tau)$. Substituting α and β we find this relationship is given by:

$$c(s,t) = K\, u(x,\tau)\, \exp\!\left(-\tfrac{1}{2}(k-1)\, x - \tfrac{1}{4}(k+1)^2\, \tau\right)$$

This allows us to conclude that the Black-Scholes equation can be expressed as the heat equation, where x is the price of the stock after transformations, τ is the unit of time after transformations, and u is the price of the option after transformations:

$$\frac{\partial u}{\partial \tau} = \frac{\partial^2 u}{\partial x^2}, \qquad -\infty < x < \infty, \qquad \tau > 0$$

Where we have the payoff boundary condition:

$$u(x,0) = \max\left\{\exp\left(\tfrac{1}{2}(k+1)\,x\right) - \exp\left(\tfrac{1}{2}(k-1)\,x\right),\ 0\right\}$$

Final Solution

We can then obtain the solutions to the Black-Scholes equation, which shows the pricing of European call and put options:

$$c = S_0 N(d_1) - K \exp(-rT)\, N(d_2) \qquad p = K \exp(-rT)\, N(-d_2) - S_0 N(-d_1)$$

where

$$d_1 = \frac{\ln\left(\frac{S_0}{K}\right) + \left(r + \frac{\sigma^2}{2}\right)(T-t)}{\sigma\sqrt{T-t}} \qquad \text{and} \qquad d_2 = d_1 - \sigma\sqrt{T-t}$$

1 Black-Scholes Equation

2 Neural Network Learning
Why Neural Networks?
Methodology

3 Experimentation and Results

4 Evaluation and Conclusion

1 Black-Scholes Equation

2 Neural Network Learning
Why Neural Networks?
Methodology

3 Experimentation and Results

4 Evaluation and Conclusion

The Analytical Solution and Its Limits

The Black-Scholes-Merton formula [1] gives a closed-form option price:

$$c = S_0 N(d_1) - Ke^{-rT} N(d_2)$$

Assumptions required:

- Constant volatility σ
- Log-normal returns
- No dividends
- Frictionless markets

Market Reality

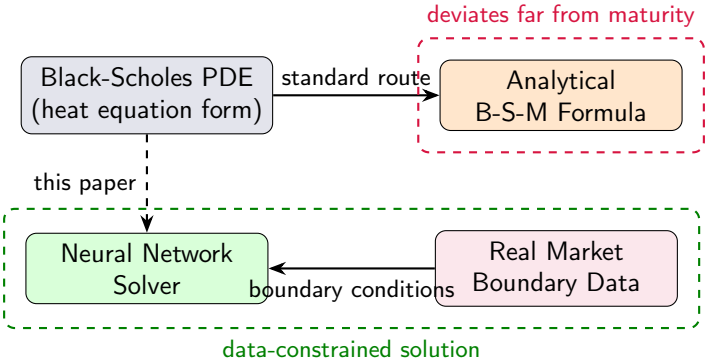
For dates **far from maturity**, the analytical solution tracks the *stock price*, not the *option price*.

It only converges to market prices near expiration.

Key Claim [3]

The issue is not the PDE itself. The real-world divergence comes from the **analytical solution** applied in practice.

PDE-Constrained Learning: The Core Idea



This is not regression

The network does not curve-fit option prices directly. It must learn a function that **satisfies the PDE** as a constraint while matching real market boundary data.

1 Black-Scholes Equation

2 Neural Network Learning
- Why Neural Networks?
- Methodology

3 Experimentation and Results

4 Evaluation and Conclusion

(A) The PDE as an Optimization Problem

Standard PDE formulation:

$$D(u) - F = 0$$

Network output $\hat{u}$: a candidate solution.

Residual:

$$\mathcal{R}(\hat{u}) = D(\hat{u}) - F$$

Training objective:

$$\min_{\theta} \|\mathcal{R}(\hat{u}_\theta)\|^2$$

When $\mathcal{R} = 0$ everywhere, $\hat{u}$ is an exact solution.

Physics-Informed Neural Networks

This is the core of **PINNs**, first proposed by Lagaris, Likas, and Fotiadis [6] and extended by Raissi, Perdikaris, and Karniadakis [8].

The PDE residual acts as a **physics-based regularizer** constraining the solution space.

For Black-Scholes: $F = 0$ (homogeneous heat equation), $D(\cdot) = \partial_\tau - \partial_{xx}$

(B) Enforcing Boundary Conditions

Problem: Minimizing the residual alone does not guarantee the network satisfies the initial condition (option payoff at expiration).

Solution: modified trial function [6, 2]:

$$\tilde{u}(x,\tau) = \underbrace{u_{\tau_0}(x)}_{\text{known payoff}} + \underbrace{\left[1 - e^{-(\tau-\tau_0)}\right]}_{\substack{=0 \text{ at } \tau=\tau_0 \\ \to 1 \text{ as } \tau \text{ grows}}} \cdot \hat{u}(x,\tau)$$

Effect

At $\tau = \tau_0$: $\tilde{u} = u_{\tau_0}(x)$ **exactly**

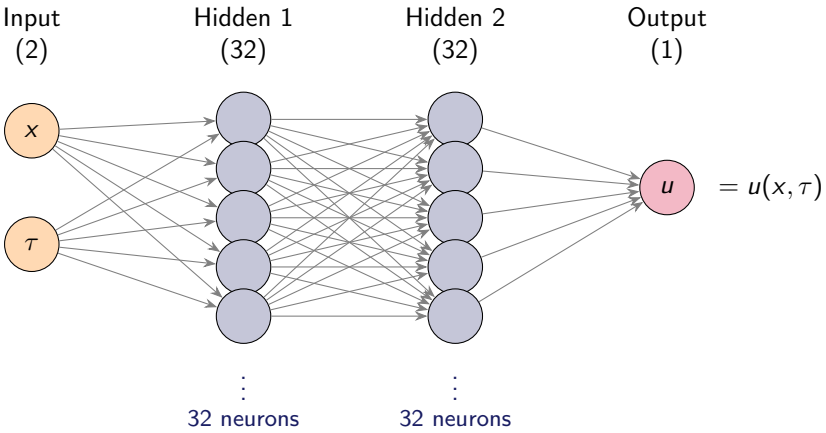
As τ grows: $\tilde{u} \approx \hat{u}(x,\tau)$

Why not a penalty term?

Penalty-based enforcement creates competing gradients. The modified trial function bakes the constraint into the architecture: satisfied *analytically*, not approximately.

General form: $\tilde{u}(x,\tau) = A(x,\tau;\, x_{\text{bdy}}, \tau_0)\, \hat{u}(x,\tau)$, where A encodes all boundary constraints. Implemented via **NeuroDiffEq** [2].

(C) Network Architecture: Structure



MLP with architecture $2 \to 32 \to 32 \to 1$ [3].
Inputs are transformed coordinates (x, τ); output is the generalised option price.

(C) Network Architecture: Training Details

Component	Choice
Architecture	MLP $(2 \to 32 \to 32 \to 1)$
Input x	$\ln(S/c)$, scaled by K
Input τ	$\frac{1}{2}\sigma^2(T-t)$
Activation	tanh
Optimizer	Adam [5]
Epochs	30,000
Framework	PyTorch [7]
Library	NeuroDiffEq [2]

Why tanh? [6, 4]

Smooth and differentiable everywhere. Computing the PDE residual requires differentiating the network output w.r.t. inputs, so the activation must be differentiable across the whole domain.

Why Adam? [5]

Adapts the learning rate per parameter using gradient moment estimates. Converges faster than vanilla SGD, especially when gradient scales vary across the network.

Coordinate Transformation: Connecting u to Real Prices

The network operates in transformed coordinates [3]. Recovering the real option price requires two substitutions:

Variable Substitutions (Black-Scholes → Heat Equation)

$$x = \ln\left(\frac{S(t)}{c(t)}\right), \qquad \tau = \tfrac{1}{2}\sigma^2(T-t)$$

$$c(S,t) = K\,\hat{u}(x,\tau)\,\exp\left(-\tfrac{1}{2}(k-1)x - \tfrac{1}{4}(k+1)^2\tau\right), \quad k = \frac{2r}{\sigma^2}$$

Inputs to the network:

- x: log-scaled, normalised price ratio
- τ: variance-scaled time to maturity

Output from the network:

- $u(x,\tau)$: generalised (transformed) price
- Back-transform gives $c(S,t)$ in BRL

1 Black-Scholes Equation

2 Neural Network Learning

3 Experimentation and Results

4 Evaluation and Conclusion

Neural Network Architecture

- **Structure:** 2-32-32-1
  - 2 inputs
  - 2 hidden layers with 32 neurons each
  - 1 output (option price)

Neural Network Architecture

- **Structure:** 2-32-32-1
  - 2 inputs
  - 2 hidden layers with 32 neurons each
  - 1 output (option price)
- **Activation Function:** Hyperbolic Tangent

Neural Network Architecture

- **Structure:** 2-32-32-1
  - 2 inputs
  - 2 hidden layers with 32 neurons each
  - 1 output (option price)
- **Activation Function:** Hyperbolic Tangent
- **Training Algorithm:** Adam optimizer

Neural Network Architecture

- **Structure:** 2-32-32-1
  - 2 inputs
  - 2 hidden layers with 32 neurons each
  - 1 output (option price)
- **Activation Function:** Hyperbolic Tangent
- **Training Algorithm:** Adam optimizer
- **Training Duration:** 30,000 epochs

Computational Framework

- Implemented in **Python 3**

Computational Framework

- Implemented in **Python 3**
- Built using the **Torch framework**

Computational Framework

- Implemented in **Python 3**
- Built using the **Torch framework**
- Utilized the **Neurodiffeq** library

Computational Framework

- Implemented in **Python 3**
- Built using the **Torch framework**
- Utilized the **Neurodiffeq** library
- Purpose: Solve differential equations using neural networks

Mathematical Methodology

- Transformed the **Black-Scholes equation** into the heat equation

Mathematical Methodology

- Transformed the **Black-Scholes equation** into the heat equation
- Change of variables:
  - Generalized price: x
  - Generalized time: τ

Mathematical Methodology

- Transformed the **Black-Scholes equation** into the heat equation
- Change of variables:
  - Generalized price: x
  - Generalized time: τ
- Applied real market payoff data as boundary conditions

Evaluation Metrics I

To evaluate model performance:

- **MAE** – Mean Absolute Error

Evaluation Metrics I

To evaluate model performance:

- **MAE** – Mean Absolute Error

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}\left|Y_i - \hat{Y}_i\right|$$

- **MSE** – Mean Square Error

Evaluation Metrics I

To evaluate model performance:

- **MAE** – Mean Absolute Error

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}\left|Y_i - \hat{Y}_i\right|$$

- **MSE** – Mean Square Error

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(Y_i - \hat{Y}_i\right)^2$$

- **MAPE** – Mean Absolute Percentage Error

Evaluation Metrics I

To evaluate model performance:

- **MAE** – Mean Absolute Error

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}\left|Y_i - \hat{Y}_i\right|$$

- **MSE** – Mean Square Error

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(Y_i - \hat{Y}_i\right)^2$$

- **MAPE** – Mean Absolute Percentage Error

$$\text{MAPE} = \frac{1}{N}\sum_{i=1}^{N}\left|\frac{Y_i - \hat{Y}_i}{Y_i}\right|$$

Evaluation Metrics II

To evaluate model performance:

- **POCID** – Prediction of Change in Direction

Evaluation Metrics II

To evaluate model performance:

- **POCID** – Prediction of Change in Direction

$$\text{POCID} = 100 \cdot \frac{\sum_{i=1}^{N} D_i}{N}$$

where

$$D_i = \begin{cases} 1, & \text{if } (Y_i - Y_{i-1})(\hat{Y}_i - \hat{Y}_{i-1}) > 0 \\ 0, & \text{otherwise} \end{cases}$$

- **ARV** – Average Relative Variance

Evaluation Metrics II

To evaluate model performance:

- **POCID** – Prediction of Change in Direction

$$\text{POCID} = 100 \cdot \frac{\sum_{i=1}^{N} D_i}{N}$$

where

$$D_i = \begin{cases} 1, & \text{if } (Y_i - Y_{i-1})(\hat{Y}_i - \hat{Y}_{i-1}) > 0 \\ 0, & \text{otherwise} \end{cases}$$

- **ARV** – Average Relative Variance

$$\text{ARV} = \frac{\sum_{i=1}^{N} (\hat{Y}_i - Y_i)^2}{\sum_{i=1}^{N} (\hat{Y}_i - \bar{Y})^2}$$

Results

The experimental results demonstrate that the Multi-Layer Perceptron (MLP) neural network successfully learned to solve the modified Black-Scholes equation, providing more accurate option pricing forecasts than traditional analytical solutions.

Table 1

TABLE 1. Statistical Errors for MLP Neural Network Modeling the Petrobras Options with Black-Scholes Model - A Series for PETR4. The best value for each error measure is in boldface.

PETRA	K (BRL)	MAE	MSE	MAPE	POCID	ARV	N
332	24.76	**0.980**	**1.295**	1.257	75.000	0.004	61
357	26.26	1.145	1.641	2.590	77.358	0.023	54
356	25.76	1.345	2.064	2.733	82.353	0.020	52
366	26.76	1.141	2.337	2.114	71.667	0.020	61
342	25.26	1.530	2.706	2.357	**84.314**	0.019	52
266	17.76	5.749	37.778	0.800	53.333	0.013	61
391	22.76	1.179	1.888	**0.528**	78.333	**0.003**	61
321	22.26	1.670	4.563	0.533	71.667	0.006	61
282	18.76	6.231	42.81	1.084	72.340	0.019	48
53	15.26	2.800	10.538	0.598	46.154	0.023	40
108	13.76	8.418	73.457	0.820	72.414	0.032	30
362	27.26	1.011	1.942	2.163	68.333	0.020	61

Figure 1

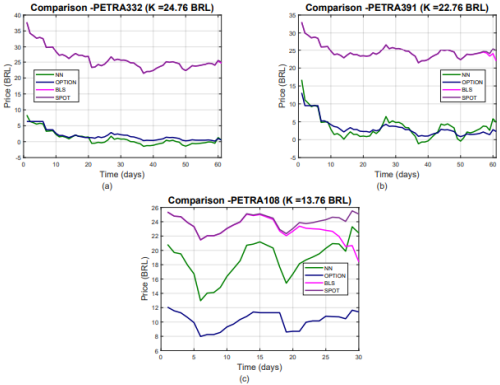


FIGURE 1. (a) Results for PETRA332 (MSE minor) (b) Results for PETRA391 and (c) results for PETRA108 (MSE major). (a) and (b) are the best NN prediction cases, and (c) is the worst NN prediction case. For all graphics, there are four curves: the price of the underlying PETR4 stock - OPTION, the blue curve; the price calculated by the Black-Scholes analytical solution - BLS, the magenta curve; the price computed by the ANN, the green curve; and, the option market price - SPOT, the purple curve.

Table 2

TABLE 2. Statistical Errors for MLP Artificial Neural Network (ANN) Modeling the Petrobras Options with Black-Scholes Model - D Serie for PETR4. The best value for each error measure is in boldface.

PETRD	K (BRL)	MAE	MSE	MAPE	POCID	ARV	N
291	28.26	0.727	0.906	1.922	16.667	0.028	61
222	26.76	1.114	1.664	3.324	8.333	0.069	61
266	27.76	**0.670**	**0.751**	2.115	11.667	0.054	61
271	27.26	0.806	1.020	2.507	13.333	0.067	61
220	26.26	1.198	1.997	2.231	10.000	0.056	61
227	25.26	1.539	3.602	0.938	10.000	0.030	61
202	25.76	1.301	2.484	1.377	6.667	0.040	61
194	23.76	2.173	6.636	**0.771**	16.667	0.024	61
183	24.76	1.798	4.599	1.089	11.667	0.036	61
198	22.76	2.848	9.541	0.855	33.333	0.019	61
122	22.26	2.751	8.501	0.822	**35.000**	**0.018**	61

Figure 2

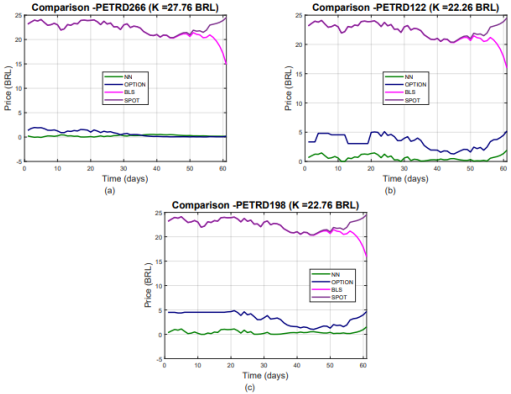


FIGURE 2. (a) Results for PETRD266 (MSE minor), (b) Results for PETRD122 and (c) results for PETRA198 (MSE major). (a) and (b) are the best NN prediction cases, and (c) is the worst NN prediction case. For all graphics, there are four curves: the price of the underlying PETR4 stock - OPTION, the blue curve; the price calculated by the Black-Scholes analytical solution - BLS, the magenta curve; the price computed by the ANN, the green curve; and, the option market price - SPOT, the purple curve.

1 Black-Scholes Equation

2 Neural Network Learning

3 Experimentation and Results

4 Evaluation and Conclusion
- Strengths
- Limitations
- Implications for Financial Modelling

1 Black-Scholes Equation

2 Neural Network Learning

3 Experimentation and Results

4 Evaluation and Conclusion
- Strengths
- Limitations
- Implications for Financial Modelling

Table 3: Interpretation of Results

- **Table 3** presents the statistical errors of the neural network across different option contracts
- The accuracy varies significantly between contracts
- Options with lower strike prices (in-the-money) generally show lower errors
- Options with higher errors tend to be less stable or less frequently traded

Key insight: The performance of the neural network depends on the characteristics of the option, particularly trading activity and moneyness.

Figure 3: Model Comparison

- **Figure 3** compares:
  - Market price (SPOT)
  - Black–Scholes (BLS)
  - Neural network (NN)
- (a): Best NN prediction (low MSE)
- (b): Worst NN prediction (high MSE)
- NN follows market trend well in (a), but deviates more in (b)

Key insight: Neural networks can closely match market prices in some cases, but performance is not consistent across all scenarios.

Evaluation and Conclusion

So far, we have seen how the Black-Scholes equation can be derived from financial principles and transformed into a PDE, and how a neural network can be trained to approximate its solution.

In this final section, we will evaluate this approach by briefly discussing its strengths, limitations, and implications for financial modelling.

Strengths of the Neural Network Approach

Recall the Black-Scholes equation:

$$\frac{\partial V(S,t)}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V(S,t)}{\partial S^2} + rS\frac{\partial V(S,t)}{\partial S} - rV(S,t) = 0$$

We want to approximate the option value $V(S,t)$. By utilising neural networks, we have the following advantages:

- Handles higher dimensions

Strengths of the Neural Network Approach

Recall the Black-Scholes equation:

$$\frac{\partial V(S,t)}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V(S,t)}{\partial S^2} + rS\frac{\partial V(S,t)}{\partial S} - rV(S,t) = 0$$

We want to approximate the option value $V(S,t)$. By utilising neural networks, we have the following advantages:

- Handles higher dimensions
- Applicable Without Closed-Form Solutions

Advantage 1: Handles Higher-Dimensional Problems

In more realistic financial models, the option price may depend on more than one variable.

For example, an option depending on two assets can be written as

$$V = V(S_1, S_2, t)$$

This leads to a larger PDE involving derivatives with respect to both asset prices.

As the number of variables increases, solving the PDE becomes more complicated. Neural networks can approximate these higher-dimensional solutions more easily.

Advantage 2: Applicable Even Without Closed-Form Solution

Not all options have a simple formula like the standard European call.

For example, an Asian option depends on the average stock price over time:

$$V = V\left(S(t), \frac{1}{T}\int_0^T S(t)dt, t\right)$$

This makes it difficult to obtain a closed-form solution.

However, the Black–Scholes PDE still applies as it gives a rule that the option price must satisfy, even for these complex cases.

1 Black-Scholes Equation

2 Neural Network Learning

3 Experimentation and Results

4 Evaluation and Conclusion
- Strengths
- Limitations
- Implications for Financial Modelling

Limitation 1: High Computational Cost

- Neural networks must be trained using many sampled points from the PDE domain

Limitation 1: High Computational Cost

- Neural networks must be trained using many sampled points from the PDE domain
- Each training step requires computing derivatives of the network output (PDE residual)

Limitation 1: High Computational Cost

- Neural networks must be trained using many sampled points from the PDE domain
- Each training step requires computing derivatives of the network output (PDE residual)
- This makes training significantly slower than standard finite difference methods for simple PDEs

Limitation 1: High Computational Cost

- Neural networks must be trained using many sampled points from the PDE domain
- Each training step requires computing derivatives of the network output (PDE residual)
- This makes training significantly slower than standard finite difference methods for simple PDEs
- For the classical Black–Scholes model, closed-form or fast numerical methods are often more efficient

Limitation 2: Harder to Interpret

- Neural networks are often described as "black-box" models because the internal decision process is not easily interpretable

Limitation 2: Harder to Interpret

- Neural networks are often described as "black-box" models because the internal decision process is not easily interpretable
- Unlike analytical Black–Scholes formulas, it is difficult to see how individual inputs (e.g. volatility or strike price) affect the output

Limitation 2: Harder to Interpret

- Neural networks are often described as "black-box" models because the internal decision process is not easily interpretable
- Unlike analytical Black–Scholes formulas, it is difficult to see how individual inputs (e.g. volatility or strike price) affect the output
- Model accuracy depends strongly on training data and network design choices

Limitation 2: Harder to Interpret

- Neural networks are often described as "black-box" models because the internal decision process is not easily interpretable
- Unlike analytical Black–Scholes formulas, it is difficult to see how individual inputs (e.g. volatility or strike price) affect the output
- Model accuracy depends strongly on training data and network design choices
- This lack of transparency can be a concern in financial settings where interpretability is important

1 Black-Scholes Equation

2 Neural Network Learning

3 Experimentation and Results

4 Evaluation and Conclusion
- Strengths
- Limitations
- Implications for Financial Modelling

Implications for Financial Modelling

- Neural networks provide a flexible framework for solving option pricing PDEs

Implications for Financial Modelling

- Neural networks provide a flexible framework for solving option pricing PDEs
- They can incorporate market data, potentially capturing effects not reflected in classical models

Implications for Financial Modelling

- Neural networks provide a flexible framework for solving option pricing PDEs
- They can incorporate market data, potentially capturing effects not reflected in classical models
- Useful for high-dimensional or complex derivatives where analytical formulas are unavailable

Implications for Financial Modelling

- Neural networks provide a flexible framework for solving option pricing PDEs
- They can incorporate market data, potentially capturing effects not reflected in classical models
- Useful for high-dimensional or complex derivatives where analytical formulas are unavailable
- Even when closed-form solutions exist, they may not fully match observed market prices

Conclusion

- The Black–Scholes equation provides a fundamental PDE framework for option pricing in financial mathematics
- While classical methods rely on analytical solutions or grid-based numerical schemes, these approaches can be limited in complexity and scalability
- Neural networks offer an alternative approach by directly approximating the PDE solution in a flexible way
- This makes them particularly useful for high-dimensional problems and financial models
- However, issues such as computational cost, interpretability, and model validation remain important in practical applications

[1] Fischer Black and Myron Scholes. "The Pricing of Options and Corporate Liabilities". In: *Journal of Political Economy* 81.3 (1973), pp. 637–654.

[2] Feiyu Chen et al. "NeuroDiffEq: A Python Package for Solving Differential Equations with Neural Networks". In: *Journal of Open Source Software* 5.46 (2020), p. 1931.

[3] Daniel de Souza Santos and Tiago Alessandro Espínola Ferreira. "Neural Network Learning of Black-Scholes Equation for Option Pricing". In: *arXiv preprint arXiv:2405.05780* (2024).

[4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[5] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *3rd International Conference on Learning Representations (ICLR)*. 2015.

[6] Isaac E. Lagaris, Aristidis Likas, and Dimitrios I. Fotiadis. "Artificial Neural Networks for Solving Ordinary and Partial Differential Equations". In: *IEEE Transactions on Neural Networks* 9.5 (1998), pp. 987–1000.

[7] Adam Paszke, Sam Gross, Francisco Massa, et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019, pp. 8024–8035.

[8] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations". In: *Journal of Computational Physics* 378 (2019), pp. 686–707.